

主要内容

- 基本语法
- 数据类型
 - null和undefined
 - 布尔值
 - 数值
 - 字符串
 - 对象
- 运算符
- 标准库

函数的参数

- 函数参数不是必需的，JavaScript 允许省略参数，省略的参数的值就变为undefined

```
>> function f(a, b) {  
    return a;  
}
```

```
← undefined
```

```
>> f(1,2,3)
```

```
← 1
```

```
>> f(1)
```

```
← 1
```

```
>> f()
```

```
← undefined
```

```
>> f.length
```

```
← 2
```

```
>> f(,1)
```

```
▲ SyntaxError: expected expression, got ',' [详细了解]
```

```
>> f(undefined, 2)
```

```
← undefined
```

函数f定义了两个参数，但是运行时无论提供多少个参数（或者不提供参数），**JavaScript 都不会报错**



函数的参数

- 函数参数不是必需的，JavaScript 允许省略参数，省略的参数的值就变为undefined

函数的length属性与实际传入的参数个数无关，**只反映函数预期传入的参数个数**

```
>> function f(a, b) {  
    return a;  
}  
← undefined  
>> f(1,2,3)  
← 1  
>> f(1)  
← 1  
>> f()  
← undefined  
>> f.length  
← 2  
>> f(,1)  
▲ SyntaxError: expected expression, got ',' [详细了解]  
>> f(undefined, 2)  
← undefined
```

函数的参数

- 函数参数不是必需的，JavaScript 允许省略参数，省略的参数的值就变为undefined

```
>> function f(a, b) {  
    return a;  
}  
← undefined  
>> f(1,2,3)  
← 1
```

没有办法只省略靠前的参数，而保留靠后的参数。如果一定要省略靠前的参数，只有显式传入undefined

```
← undefined
```

```
>> f.length
```

```
← 2
```

```
>> f(,1)
```

```
⚠ SyntaxError: expected expression, got ',' [详细了解]
```

```
>> f(undefined, 2)
```

```
← undefined
```

函数的参数

- 如果有同名的参数，则取最后出现的那个值，即使后面的没有值或被省略，也是以其为准

```
function f(a, a) {  
  console.log(a);  
}
```

```
f(1, 2) // 2
```

```
function f(a, a) {  
  console.log(a);  
}
```

```
f(1) // undefined
```

- 函数f有两个参数，且参数名都是a，取值的时候，以后面的a为准
- 如果调用函数f的时候没有提供第二个参数，a的取值就变成了undefined

参数的两种传递方式

- 函数参数如果是原始类型的值（数值、字符串、布尔值），传递方式是传值传递（passes by value），在函数体内修改参数值，不会影响到原始值
- 如果函数参数是复合类型的值（数组、对象、其他函数），传递方式是传址传递（pass by reference），传入函数的是原始值的地址，因此在函数内部修改参数，将会影响到原始值

参数的两种传递方式

```
var p = 2;

function f(p) {
  p = 3;
}
f(p);

p // 2
```

VS

```
var obj = { p: 1 };

function f(o) {
  o.p = 2;
}
f(obj);

obj.p // 2
```

传值

把外面 **p** 的 **值 2** 拷贝一份，传给函数参数 **p**
函数内部的 **p** 现在是一个 **局部变量**，值是 2
执行 **p = 3;** 只是把 **局部变量 p** 改成 3
函数结束后，这个局部 **p** 就销毁了，外面的全局 **p** 根本没动

传地址

传入函数的原始值的地址，因此在函数内部修改参数，将会影响到原始值

参数的两种传递方式

- 如果函数内部修改的，不是参数对象的某个属性，而是把参数对象整个替换成另一个值，这时**不会影响到原始值**

```
>> var obj = [1, 2, 3];  
  
    function f(o) {  
      o = [2, 3, 4];  
    }  
    f(obj);  
← undefined  
  
>> obj  
← ► Array(3) [ 1, 2, 3 ]
```

形式参数（o）的值实际是参数obj的**地址副本**，重新对o赋值导致o指向另一个地址，保存在原地址上的值不受影响

arguments 对象

- arguments对象只在函数体内部才可以使用，包含了函数运行时的所有参数，可以在函数体内部读取所有参数，通过arguments对象的length属性，可以判断函数调用时到底带几个参数

```
>> var f = function (one) {  
    console.log(arguments.length+" arguments:")  
    console.log("arg0: "+ arguments[0]);  
    console.log("arg1: "+ arguments[1]);  
    console.log("arg2: "+ arguments[2]);  
}  
← undefined  
>> f(1,2,3,4,5)  
5 arguments:  
arg0: 1  
arg1: 2  
arg2: 3  
← undefined  
>> f(1,2)  
2 arguments:  
arg0: 1  
arg1: 2  
arg2: undefined  
← undefined
```

虽然只声明了一个形参 one
但 arguments 会装入所有实际传入的参数
arguments.length 是传入参数的个数

arguments 对象

- 正常模式下，arguments对象可以在运行时修改

```
var f = function(a, b) {  
  arguments[0] = 3;  
  arguments[1] = 2;  
  return a + b;  
}  
  
f(1, 1) // 5
```

- 严格模式**下arguments对象是一个只读对象，修改它是无效的，但不会报错

```
var f = function(a, b) {  
  'use strict'; // 开启严格模式  
  arguments[0] = 3; // 无效  
  arguments[1] = 2; // 无效  
  return a + b;  
}  
  
f(1, 1) // 2
```

在JavaScript中使用"use strict"指令时，会启用严格模式。严格模式是一种更加严格的JavaScript解析和错误处理方式，它有助于减少一些常见的错误，并使代码更具健壮性。

1. 禁止使用八进制数字字面量（如0123）。
2. 禁止使用with语句。

变量必须先声明后使用。未声明的变量将会抛出错误。

闭包 (Closures)

- 闭包 (closure) 是 JavaScript 语言的一个难点，也是它的特色，很多高级应用都要依靠闭包实现
- 闭包是函数和声明该函数的词法环境的组合

闭包 (Closures)

- 问题提出：做一个函数，这个函数每执行一次就累积减一

```
<script>
  var a = 10;
  function fa(){
    a--;
    console.log(a);
  }
</script>
```

采用全局变量存在的问题：

- 1、占用内存
- 2、存在变量污染问题(使用了相同的标识符声明了全局变量，var关键字声明相同的变量名会覆盖，let、const重复声明相同的变量名会直接报错；)

闭包 (Closures)

- 问题提出：做一个函数，这个函数每执行一次就累积减一

```
function fa(){  
  let a = 10;  
  a--;  
  console.log(a);  
}
```

let 声明的变量的作用域则是它当前所处代码块，即它的作用域既可以是全局或者整个函数块，也可以是 if、while、switch 等用 {} 限定的代码区块

这样就可以了。。。

吗？

每次函数一调用完，就会销毁，a 永远都是 10 开始，存不住

闭包 (Closures)

- 问题提出：做一个函数，这个函数每执行一次就累积减一

```
function fa(){  
  let a = 10;  
  function fb(){  
    a--;  
    console.log(a);  
  }  
  return fb;  
}  
var fm = fa();
```

```
> fm();  
9  
< undefined  
> fm();  
8  
< undefined  
> fm();  
7  
< undefined  
>
```

- 1、有函数嵌套
- 2、内部函数引用外部作用域的变量参数
- 3、返回值是函数
- 4、创建一个对象函数，让其长期驻留

fa() 执行结束，按理 a 应该被销毁

但因为 fb 还在用 a，所以 JavaScript 不会销毁 a

a 被“闭合”在 fb 的词法作用域里——这就是闭包的本质，只要内部函数在外部被继续使用并且仍然访问着外部函数的变量，就形成闭包

我们需要一种机制，即能长期保存变量又不污染全局，这就是闭包

即使 fa 已经执行完毕，a 却继续存在、继续减少、继续被访问。

立即调用的函数表达式 (IIFE)

- 在 Javascript 中，圆括号()是一种运算符，跟在函数名之后，表示调用该函数

```
>> function f(){console.log(".....");}  
← undefined  
>> f()  
.....  
← undefined
```

- 有时，我们需要在定义函数之后，立即调用该函数，不能在函数的定义之后加上圆括号，这会产生语法错误

```
>> function f(){console.log(".....");}  
← undefined  
>> function f(){console.log(".....");}();  
⚠ SyntaxError: expected expression, got ')'  
[详细了解]
```

立即调用的函数表达式 (IIFE)

- 在JavaScript中，function这个关键字即可以当作语句，也可以当作表达式
- 为了避免解析上的歧义，JavaScript引擎规定，如果function关键字出现在行首，一律解释成语句
- JavaScript引擎看到行首是function关键字之后，认为这一段都是函数的定义，不应该以圆括号结尾，所以就报错了

```
function add(a, b) {  
    return a + b;  
}
```

```
var add = function(a, b) {  
    return a + b;  
};
```


立即调用的函数表达式 (IIFE)

- 解决方法就是不要让function出现在行首，让引擎将其理解成一个表达式，最简单的处理就是将其放在一个圆括号里面，引擎就会认为后面跟的是一个表示式，而不是函数定义语句，所以就避免了错误
- 这就叫做“立即调用的函数表达式” (Immediately-Invoked Function Expression)，简称 IIFE

```
(function(){ /* code */ })();  
// 或者  
(function(){ /* code */ })();
```

立即调用的函数表达式 (IIFE)

- 推而广之，任何让解释器以表达式来处理函数定义的方法，都能产生立即调用的效果

```
var i = function(){ return 10; }();  
true && function(){ /* code */ }();  
0, function(){ /* code */ }();
```

逗号运算符，会依次执行左边和右边的表达式，并返回右边表达式的值。

```
!function () { /* code */ }(); 取反  
~function () { /* code */ }(); 按位非  
-function () { /* code */ }(); 取负值  
+function () { /* code */ }(); 取正值
```

总结：立即调用的函数表达式 (IIFE)：

- ① function关键字不出现在行首，让解释器认为是表达式
- ② 函数定义后加上()，调用函数

数组的定义

- 数组（array）是**按次序排列**的一组值。每个值的位置都有编号（从0开始），整个数组用方括号表示

```
>> var arr = [];  
    arr[0] = 1;  
    arr[1] = {a:1};  
    arr[2] = [1,2,3];  
    arr[3] = function(){console.log(".....")};  
← ▶ function 3()  
>> arr[0]  
← 1  
>> arr[1].a  
← 1  
>> arr[2][2]  
← 3  
>> (arr[3])()  
.....
```

说明：

- ① 除了在设计时赋值，数组也可以先定义后赋值
- ② 任何类型的数据，都可以放入数组（数值、对象、数组、函数）
- ③ 如果数组的元素还是数组，就形成了多维数组

数组的本质

- 本质上，数组属于一种特殊的对象。typeof运算符会返回数组的类型是object，数组的特殊性体现在，它的键名是**按次序排列**的一组整数

```
>> var arr = ['a','b','c'];  
< undefined  
>> typeof arr  
< "object"  
>> Object.keys(arr)  
< ▶ Array(3) [ "0", "1", "2" ]  
>> Object.values(arr)  
< ▶ Array(3) [ "a", "b", "c" ]  
>> arr['0'] == arr[0]  
< true
```

由于数组成员的键名是固定的（默认总是0、1、2...），因此数组不用为每个元素指定键名，而对象的每个成员都必须指定键名

JavaScript 语言规定，**对象的键名一律为字符串**，所以**数组的键名其实也是字符串**。之所以可以用数值读取，是因为非字符串的键名会被转为字符串

数组的length属性

- 数组的length属性返回数组的成员数量，可以通过修改length属性值随时增减数组的成员

```
>> var arr = ['a','b','c'];
< undefined
>> arr.length
< 3
>> arr[5] = 'e';
< "e"
>> arr
< ► Array(6) [ "a", "b", "c", <2 empty slots>, "e" ]
>> arr[3]
< undefined
>> arr[4]
< undefined
>> arr.length
< 6
>> arr.length=2
< 2
>> arr
< ► Array [ "a", "b" ]
>> arr.length=0
< 0
>> arr
< ► Array []
```

说明:

- ① 数组的数字键不需要连续，length属性的值总是比最大的那个整数键大1
- ② 数组是一种动态的数据结构，可以随时增减数组的成员
- ③ length属性是可写的，当length属性设为大于数组个数时，读取新增的位置都会返回undefined
- ④ 可以通过修改length属性值为0来删除所有的数组成员
- ⑤ JavaScript 使用一个32位整数保存数组的元素个数，length属性的最大值就是4294967295 ($=2^{32} - 1$)

数组的length 属性

- 由于数组本质上是一种对象，所以可以为数组添加属性，但是这不影响length属性的值

```
>> var a = [];  
    a['p'] = 'abc';  
    a[2.1] = 'abc';  
    a[-1] = 'abc';  
  
← "abc"  
  
>> a.length  
← 0  
  
>> a['p']  
← "abc"
```

示例将数组的键分别设为字符串和小数，结果都不影响length属性

length属性的值就是等于最大的数字键加1，而这个数组没有整数键，所以length属性保持为0

当你使用字符串作为键添加元素时，它们实际上并不算作数组的元素，而是作为对象的属性添加到数组对象中。

数组的长度只统计数组的数值索引元素的个数，而不包括非数值索引的属性。

in运算符

- 可以用运算符in检查某个键名是否存在，如果数组的某个位置是空位，in运算符返回false
- in适用于对象也适用于数组

```
>> var arr = [];  
    arr[3] = 'a';  
    arr  
  
← ▶ Array(4) [ <3 empty slots>, "a" ]  
  
>> for ( i = 0; i < arr.length; i++){  
    if ( i in arr)  
        console.log(i+" is in the array");  
    else  
        console.log(i+" isn't in the array");  
}
```

```
0 isn't in the array  
1 isn't in the array  
2 isn't in the array  
3 is in the array
```

数组的0, 1, 2
位置都是空的

数组的遍历

- `for...in`循环，不仅会遍历数组所有数字键，还会遍历非数字键，**不推荐使用`for...in`遍历数组**
- 数组的遍历可以考虑使用`for`循环或`while`循环
- 数组的`forEach`方法也可以用来遍历数组

```
>> var a = [1, 2, 3];  
    a.foo = true;  
    for (var key in a) {  
        console.log(key);  
    }  
  
0  
1  
2  
foo
```

for...in循环

数组的遍历

- **for...in**循环，不仅会遍历数组所有数字键，还会遍历非数字键，**不推荐使用for...in遍历数组**
- 数组的遍历可以考虑使用**for**循环或**while**循环
- 数组的**forEach**方法也可以用来遍历数组

```
>> var a = [1, 2, 3];  
    a.foo = true;  
  
← true  
>> for(var i = 0; i < a.length; i++) {  
    console.log(a[i]);  
}  
  
1  
2  
3
```

for循环

```
>> var a = [1, 2, 3];  
    a.foo = true;  
  
    var l = a.length;  
    while (l--) {  
        console.log(a[l]);  
    }  
  
3  
2  
1
```

while循环

数组的遍历

- 数组的forEach方法也可以用来遍历数组

```
var arr = [1, 2, 3];  
arr.forEach(function(element, index, array) {  
    console.log("元素: " + element);  
    console.log("索引: " + index);  
    console.log("数组: " + array);  
});
```

元素: 1
索引: 0
数组: [1, 2, 3]
元素: 2
索引: 1
数组: [1, 2, 3]
元素: 3
索引: 2
数组: [1, 2, 3]

forEach() 可以用来遍历数组中的每个元素，并对每个元素执行一个回调函数。forEach() 方法的参数是一个匿名函数，这个函数接受一个参数 array，表示当前正在遍历的数组元素。

在 JavaScript 中，回调函数是一种特殊的函数，它作为参数传递给另一个函数，并且在某个特定事件发生或条件满足时被调用执行。

回调函数常见于异步编程，用于处理异步操作的结果或通知。当异步操作完成时，回调函数被调用，以便处理结果或执行其他逻辑。

数组的空位

- 当数组的某个位置是空元素，即两个逗号之间没有任何值，我们称该数组存在空位（hole），数组的空位不影响length属性
- 如果最后一个元素后面有逗号，不会产生空位

```
>> var a = [1, ,1,];  
← undefined  
>> a.length  
← 3  
>> a  
← ► Array(3) [ 1, <1 empty slot>, 1 ]
```

数组的空位

- 数组的空位是可以读取的，返回undefined
- 使用**delete命令**删除一个数组成员，会形成空位，并且**不会影响length属性**

```
>> var a = [ , , ];  
    a[1]  
    < undefined  
>> var a = [1,2 , ,4];  
    < undefined  
>> a[2]  
    < undefined  
>> delete a[0]  
    < true  
>> a  
    < ▶ Array(4) [ <1 empty slot>, 2, <1 empty slot>, 4 ]  
>> a.length  
    < 4
```

数组的空位

- 数组的某个位置是空位，与某个位置是undefined，是不一样的；空位就是数组没有这个元素，所以不会被遍历到，而undefined则表示数组有这个元素，值是undefined，所以遍历不会跳过

```
var a = [, , ,];

a.forEach(function (x, i) {
  console.log(i + '. ' + x);
})
// 不产生任何输出

for (var i in a) {
  console.log(i);
}
// 不产生任何输出

Object.keys(a)
// []
```

```
var a = [undefined, undefined, undefined];

a.forEach(function (x, i) {
  console.log(i + '. ' + x);
});
// 0. undefined
// 1. undefined
// 2. undefined

for (var i in a) {
  console.log(i);
}
// 0
// 1
// 2

Object.keys(a)
// ['0', '1', '2']
```

主要内容

- 基本语法
- 数据类型
- 运算符
- 标准库

运算符

- 算术运算符
- 比较运算符
- 布尔运算符
- 二进制位运算符
- 其他运算符，运算顺序

算术运算符

- JavaScript 共提供10个算术运算符，用来完成基本的算术运算
 - 加法运算符: $x + y$
 - 减法运算符: $x - y$
 - 乘法运算符: $x * y$
 - 除法运算符: x / y
 - 指数运算符: $x ** y$
 - 余数运算符: $x \% y$
 - 自增运算符: $++x$ 或者 $x++$
 - 自减运算符: $--x$ 或者 $x--$
 - 数值运算符: $+x$
 - 负数值运算符: $-x$

加减乘除

- 加法运算符:

- 数字+数字: $1+1=2$

- 数字+布尔值:

- 布尔值都会自动转成数值, 然后再相加

- $1+\text{true} = 2$ $1 + \text{false} = 1$ $\text{null}+1 = 1$

- 字符串+字符串:

- 两个字符串相加, 这时加法运算符会变成连接运算符, 返回一个新的字符串, 将两个原字符串连接在一起

- $'a'+'bc' = 'abc'$

- 非字符串+字符串:

- 如果一个运算符是字符串, 另一个运算符是非字符串, 这时非字符串会转成字符串, 再连接在一起

- $1 + 'a' // "1a"$ $\text{false} + 'a' // "falsea"$ $1 + '5' // "15"$

对象相加

- 如果运算符是对象，必须先转成原子类型的值，然后再相加

```
var obj = { p: 1 };  
obj + 2 // "[object Object]2"
```

对象obj转成原子类型的值是字符串'[object Object]'，再加2就得到了上面的结果

- 对象转成原子类型的值计算如下：

```
var obj = { p: 1 };  
obj.valueOf().toString() // "[object Object]"
```

余数运算符

- 余数运算符 (%) 返回前一个运算符被后一个运算符除所得的余数
- 运算结果的正负由第一个运算符的正负号决定

```
>> -1 % 2  
← -1  
  
>> 1 % 2  
← 1
```

$$\begin{array}{r} > 1\% -2 \\ \hline < 1 \end{array}$$

自增和自减运算符

- 自增(++)和自减(--)运算符是一元运算符，作用是将运算符首先转为数值，然后加上1或者减去1，**它们会修改原始变量**

```
>> var x = 1, y = 1;
< undefined
>> console.log(x++)
1
< undefined
>> console.log(x)
2
< undefined
>> console.log(++y)
2
< undefined
>> console.log(y)
2
```

数值运算符，负数值运算符

- 数值运算符 (+)、负数值运算符 (-) 的作用在于可以将任何值转为数值（与Number函数的作用相同），会返回一个新的值，而**不会改变原始变量的值**

```
>> +true
< 1
>> +[]
< 0
>> +{}
< NaN
>> -false
< -0
>> +false
< 0
```

空数组转换为字符串后是空字符串，再将空字符串转换为数字时会得到 0

空对象在转换为字符串时会变成 "[object Object]"

```
>> var x = 1
< undefined
>> -x
< -1
>> x
< 1
```

指数运算符

- 指数运算符 (**) 完成指数运算，前一个运算符是底数，后一个运算符是指数
- 指数运算符是**右结合**，多个指数运算符连用时，先进行最右边的计算

```
>> 2**4
< 16
>> 2**3**2 == 2**9
< true
>> (2**3)**2
< 64
>> 2**3**2
< 512
```

赋值运算符

- 赋值运算符（Assignment Operators, =）用于给变量赋值

```
// 等同于 x = x + y  
x += y
```

```
// 等同于 x = x - y  
x -= y
```

```
// 等同于 x = x * y  
x *= y
```

```
// 等同于 x = x / y  
x /= y
```

```
// 等同于 x = x % y  
x %= y
```

```
// 等同于 x = x ** y  
x **= y
```

赋值运算符还可以与其他运算符结合，先进行指定运算，然后将得到值返回给左边的变量

比较运算符

- 比较运算符用于比较两个值的大小，**然后返回一个布尔值**，表示是否满足指定的条件

注意，比较运算符可以比较各种类型的值，不仅仅是数值。

比较运算符

- JavaScript 一共提供了8个比较运算符，分成两类：相等比较和非相等比较

- `>` 大于运算符
- `<` 小于运算符
- `<=` 小于或等于运算符
- `>=` 大于或等于运算符
- `==` 相等运算符
- `===` 严格相等运算符
- `!=` 不相等运算符
- `!==` 严格不相等运算符

非相等比较基本规则

- 非相等比较基本规则

- 先看两个运算符是否都是字符串，如果是，就按照字典顺序比较（实际上是**比较 Unicode 码点**）
- 否则，则将两个运算符都转成数值，再比较数值的大小

非相等运算符：字符串的比较

- JavaScript 引擎内部首先比较首字符的 Unicode 码点，如果相等，再比较第二个字符的 Unicode 码点，以此类推，由于所有字符都有 Unicode 码点，因此汉字也可以比较

```
>> 'cat' > 'dog'
← false
>> 'cat' > 'catdog'
← false
>> 'cat' < 'Cat'
← false
>> '大' > '小'
← false
```

```
> 'cat' > 'Cat'
< true

> '大' < '小'
< true
```

小写的c的 Unicode 码点（99）大于大写的C的 Unicode 码点（67）

“大”的 Unicode 码点是22823，
“小”是23567

非相等运算符：非字符串的比较

- 如果两个运算符都是原始类型的值，则是先转成**数值**再比较
- 任何值（包括NaN本身）与NaN比较，返回的都是false

```
5 > '4' // true
// 等同于 5 > Number('4')
// 即 5 > 4

true > false // true
// 等同于 Number(true) > Number(false)
// 即 1 > 0

2 > true // true
// 等同于 2 > Number(true)
// 即 2 > 1
```

```
1 > NaN // false
1 <= NaN // false
'1' > NaN // false
'1' <= NaN // false
NaN > NaN // false
NaN <= NaN // false
```

JavaScript 的比较操作符会根据操作数的类型来自动处理，但开发人员应该明确操作数的类型，以避免产生预期之外的结果。

```
> 5 > 'cat'
< false
> 5 < 'cat'
< false
> 5 == 'cat'
< false
> '5' < 'cat'
< true
```

非相等运算符：非字符串的比较

- 如果运算符是对象，会转为原子类型的值，再进行比较
- 对象转换成原子类型的值，算法是先调用 `valueOf` 方法；如果返回的还是对象，再接着调用 `toString` 方法（`valueOf` 方法用于返回指定对象的原始值，若对象没有原始值，则将返回对象本身。）

```
var x = [2];
x > '11' // true
// 等同于 [2].valueOf().toString() > '11'
// 即 '2' > '11'

x.valueOf = function () { return '1' };
x > '11' // false
// 等同于 [2].valueOf() > '11'
// 即 '1' > '11'
```

```
> '2' > '11'
< true
> 2 > 11
< false
```

```
>> var x = [2];
< undefined
>> x > '11'
< true
>> [2].valueOf()
< ► Array [ 2 ]
>> [2].valueOf().toString()
< "2"
```

非相等运算符：非字符串的比较

- 如果运算符是对象，会转为原子类型的值，再进行比较
- 对象转换成原子类型的值，算法是先调用valueOf方法；如果返回的还是对象，再接着调用toString方法

两个对象之间的比较

```
[2] > [1] // true
// 等同于 [2].valueOf().toString() > [1].valueOf().toString()
// 即 '2' > '1'

[2] > [11] // true
// 等同于 [2].valueOf().toString() > [11].valueOf().toString()
// 即 '2' > '11'

{ x: 2 } >= { x: 1 } // true
// 等同于 { x: 2 }.valueOf().toString() >= { x: 1 }.valueOf().toString()
// 即 '[object Object]' >= '[object Object]'
```

JavaScript 中的“相等”

- JavaScript 提供两种相等运算符：==和===。
- 它们的区别是
 - 相等运算符（==）比较两个值是否相等，严格相等运算符（===）比较它们是否为“同一个值”
 - 如果两个值不是同一类型，严格相等运算符（===）直接返回false，而相等运算符（==）会将它们转换成同一个类型，再用严格相等运算符进行比较

严格相等运算符

- 如果两个值的类型不同，直接返回false

```
1 === "1" // false  
true === "true" // false
```

- 同一类型的原始类型的值（数值、字符串、布尔值）比较时，值相同就返回true，值不同就返回false

```
1 === 0x1 // true
```

```
NaN === NaN // false  
+0 === -0 // true
```

注意特殊情况!

严格相等运算符

- 两个复合类型（对象、数组、函数）的数据比较时，不是比较它们的值是否相等，而是比较它们**是否指向同一个地址**

```
{ } === { } // false  
[ ] === [ ] // false  
(function () { } ) === function () { } // false
```

运算符两边的空对象、空数组、空函数的值，都存放在不同的内存地址，结果是false

```
var v1 = { };  
var v2 = v1;  
v1 === v2 // true
```

如果两个变量引用同一个对象，则它们相等

严格相等运算符

- 对于两个对象的比较，严格相等运算符和相等运算符比较的是地址，而大于或小于运算符比较的是值

```
>> var o1 = {};  
    var o2 = {};
```

```
< undefined
```

```
>> o1 > o2
```

```
< false
```

```
>> o1.valueOf().toString()
```

```
< "[object Object]"
```

```
>> o2.valueOf().toString()
```

```
< "[object Object]"
```

```
>> o1 === o2
```

```
< false
```

```
>> o1 == o2
```

```
< false
```

`>=` 是按“不是小于”定义的

相同类型，采用严格相等比较，地址不一致，所以为false

O1 和 O2 指向两个不同的对象

```
> var o1 = {};  
   var o2 = {};  
   o1>=o2
```

```
< true
```

```
> var o1 = {};  
   var o2 = {};  
   o1>o2
```

```
< false
```

```
> var o1 = {};  
   var o2 = {};  
   o1==o2
```

```
< false
```

严格相等运算符

- undefined和null与自身严格相等

```
undefined === undefined // true  
null === null // true
```

严格不相等运算符

- 严格相等运算符有一个对应的“严格不相等运算符” (`!==`)，它的算法就是先求严格相等运算符的结果，然后返回相反值

```
1 !== '1' // true  
// 等同于  
!(1 === '1')
```

相等运算符

- 比较相同类型的数据时，与严格相等运算符完全一样
- 比较不同类型的数据时，先将数据进行类型转换，然后再用严格相等运算符比较

```
>> 1 == 1.0
← true

>> 1 === 1.0
← true

>> var x = [];
    var y = [];

← undefined

>> x == y
← false
```

相等运算符

- 原始类型的值会转换成数值再进行比较

```
1 == true // true  
// 等同于 1 === Number(true)
```

```
0 == false // true  
// 等同于 0 === Number(false)
```

```
2 == true // false  
// 等同于 2 === Number(true)
```

```
2 == false // false  
// 等同于 2 === Number(false)
```

```
'true' == true // false  
// 等同于 Number('true') === Number(true)  
// 等同于 NaN === 1
```

```
'' == 0 // true  
// 等同于 Number('') === 0  
// 等同于 0 === 0
```

```
'' == false // true  
// 等同于 Number('') === Number(false)  
// 等同于 0 === 0
```

```
'1' == true // true  
// 等同于 Number('1') === Number(true)  
// 等同于 1 === 1
```

```
'\n 123 \t' == 123 // true  
// 因为字符串转为数字时, 省略前置和后置的空格
```

相等运算符会尝试将字符串转换为数字, 然后再进行比较。在进行转换时, 字符串会被忽略开头和结尾的空格, 以及其他空白字符 (例如换行符 `\n` 和制表符 `\t`), 只保留有效的数字部分。

相等运算符

- 对象（这里指广义的对象，包括数组和函数）与原始类型的值比较时，对象转换成原始类型的值，再进行比较

```
// 对象与数值比较时，对象转为数值
[1] == 1 // true
// 等同于 Number([1]) == 1

// 对象与字符串比较时，对象转为字符串
[1] == '1' // true
// 等同于 String([1]) == '1'
[1, 2] == '1,2' // true
// 等同于 String([1, 2]) == '1,2'

// 对象与布尔值比较时，两边都转为数值
[1] == true // true
// 等同于 Number([1]) == Number(true)
[2] == true // false
// 等同于 Number([2]) == Number(true)
```

相等运算符

- **undefined**和**null**与其他类型的值比较时，**结果都为false**，它们互相比较时结果为**true**

```
false == null // false
false == undefined // false

0 == null // false
0 == undefined // false

undefined == null // true
```

```
> 0==Number(null)
< true

> 0==null
< false
```


相等运算符

- 相等运算符隐藏的类型转换，会带来一些违反直觉的结果，很容易出错，建议使用严格相等

```
0 == ''           // true
0 == '0'          // true
```

```
2 == true         // false
2 == false        // false
```

```
false == 'false'  // false
false == '0'      // true
```

```
false == undefined // false
false == null       // false
null == undefined   // true
```

```
'\t\r\n' == 0      // true
```

Number("") 的结果是 0 (制表符\t 回车\r 换行\n)

不相等运算符

- 相等运算符有一个对应的“不相等运算符”(`!=`)，它的算法就是先求相等运算符的结果，然后返回相反值

```
1 != '1' // false
```

```
// 等同于
```

```
!(1 == '1')
```

布尔运算符

- 布尔运算符用于将表达式转为布尔值，一共包含四个运算符

- 取反运算符： `!`
- 且运算符： `&&`
- 或运算符： `||`
- 三元运算符： `?:`

取反运算符

- 取反运算符是一个感叹号!，用于将布尔值变为相反值，对于非布尔值，取反会将其转为布尔值

除了以下六个值取反后为true，其他值都为false

- undefined
- null
- false
- 0
- NaN
- 空字符串 ('')

```
!undefined // true
!null // true
!0 // true
!NaN // true
!"" // true

!54 // false
!'hello' // false
![] // false
!{} // false
```

且运算符 (&&)

- 且运算符 (&&) 往往用于多个表达式的求值，它的运算规则是：
 - 如果第一个运算符的布尔值为true，则返回第二个运算符的值（注意是值，不是布尔值）
 - 如果第一个运算符的布尔值为false，则直接返回第一个运算符的值，且不再对第二个运算符求值

```
't' && '' // ""  
't' && 'f' // "f"  
't' && (1 + 2) // 3  
'' && 'f' // ""  
'' && '' // ""
```

} 第一个运算符的布尔值为true

} 第一个运算符的布尔值为false

```
var x = 1;  
(1 - 1) && ( x += 1) // 0  
x // 1
```

第一个运算符的布尔值为false，不对第二个运算符求值
这种跳过第二个运算符的机制，被称为“短路”

且运算符 (&&)

- 且运算符可以多个连用，这时返回第一个布尔值为false的表达式的值。如果所有表达式的布尔值都为true，则返回最后一个表达式的值

```
>> true && 'foo' && NaN && 4 && 'foo' && true  
← NaN  
  
>> 1 && 2 && (7+8)  
← 15
```

或运算符 (||)

- 或运算符 (||) 也用于多个表达式的求值。它的运算规则是：
 - 如果第一个运算符的布尔值为true，则返回第一个运算符的值，且不再对第二个运算符求值
 - 如果第一个运算符的布尔值为false，则返回第二个运算符的值
- 短路规则对或运算符也适用
 - 这种只通过第一个表达式的值，控制是否运行第二个表达式的机制，就称为“短路” (short-cut)

三元条件运算符 (?:)

- 三元条件运算符由问号 (?) 和冒号 (:) 组成，分隔三个表达式，如果第一个表达式的布尔值为true，则返回第二个表达式的值，否则返回第三个表达式的值

```
't' ? 'hello' : 'world' // "hello"  
0 ? 'hello' : 'world' // "world"
```


二进制位运算符

- 二进制位运算符用于直接对二进制位进行计算，一共有7个
 - ① 二进制或运算符 (or) : 符号为|
 - ② 二进制与运算符 (and) : 符号为&
 - ③ 二进制否运算符 (not) : 符号为~
 - ④ 异或运算符 (xor) : 符号为^
 - ⑤ 左移运算符 (left shift) : <<
 - ⑥ 右移运算符 (right shift) : >>
 - ⑦ 无符号右移运算符: >>>

数据类型的转换

- JavaScript 是一种动态类型语言，变量没有类型限制，可以随时赋予任意值

```
var x = y ? 1 : 'a';
```

x的类型没法在编译阶段就知道，
必须等到运行时才能知道

- 虽然变量的数据类型是不确定的，但是**各种运算符对数据类型是有要求的**，如果运算符发现运算子的类型与预期不符，就会自动转换类型
- 学习数据类型自动转换的规则对于正确理解运算结果十分重要

强制类型转换

- 使用Number()将各种类型的值强制转换成数字，
— 参数是原始类型的值，规则如下：

```
// 数值：转换后还是原来的值
Number(324) // 324

// 字符串：如果可以被解析为数值，则转换为相应的数值
Number('324') // 324

// 字符串：如果不可以被解析为数值，返回 NaN
Number('324abc') // NaN

// 空字符串转为0
Number('') // 0
```

```
// 布尔值：true 转成 1, false 转成 0
Number(true) // 1
Number(false) // 0

// undefined：转成 NaN
Number(undefined) // NaN

// null：转成0
Number(null) // 0
```

```
parseInt('42 cats') // 42
Number('42 cats') // NaN
```

Number函数将字符串转为数值，**要比parseInt函数严格很多**，parseInt逐个解析字符，而Number函数整体转换字符串的类型，只要有一个字符无法转成数值，整个字符串就会被转为NaN

强制类型转换

- 使用Number()将各种类型的值强制转换成数字，
 - 参数是对象，规则如下：
 - 返回NaN，除非是包含0个或者单个数值的数组

```
>> Number({a: 1})  
← NaN  
>> Number([1,2,3])  
← NaN  
>> Number(function() {})  
← NaN  
>> Number([1])  
← 1  
>> Number([])  
← 0
```

强制类型转换

- 使用**String()**将任意类型的值强制转换成字符串
 - 参数是原始类型值，转换规则如下：
 - 数值：转为相应的字符串
 - 字符串：转换后还是原来的值
 - 布尔值：**true**转为字符串"**true**"，**false**转为字符串"**false**"
 - **undefined**：转为字符串"**undefined**"
 - **null**：转为字符串"**null**"

```
String(123) // "123"  
String('abc') // "abc"  
String(true) // "true"  
String(undefined) // "undefined"  
String(null) // "null"
```

强制类型转换

- 使用String()将任意类型的值强制转换成字符串

- 参数是对象，转换规则如下：

- 参数如果是狭义对象，返回一个类型字符串
- 参数如果是函数，返回函数.toString值
- 参数如果是数组，返回该数组的字符串形式

```
>> String({a: 1})  
← "[object Object]"  
  
>> function f(){ return null;}  
← undefined  
  
>> String(f)  
← "function f(){ return null;}"  
  
>> f.toString()  
← "function f(){ return null;}"  
  
>> String([1,2,3])  
← "1,2,3"
```

强制类型转换

- 使用 **Boolean()** 将输入的各种类型的值强制转换成布尔值
 - 转换规则相对简单：除了以下五个值的转换结果为 **false**，其他的值全部为 **true**
 - undefined
 - null
 - -0 或 +0
 - NaN
 - "" (空字符串)

```
Boolean(undefined) // false
Boolean(null) // false
Boolean(0) // false
Boolean(NaN) // false
Boolean('') // false
```

自动类型转换

- 自动转换是以强制转换为基础的，遇到以下三种情况时，JavaScript 会自动转换数据类型，即转换是自动完成的，用户不可见
 - 第一种情况，不同类型的数据互相运算
 - 第二种情况，对非布尔值类型的数据求布尔值
 - 第三种情况，对非数值类型的值使用一元运算符（即+和-）
- 自动转换的规则是：预期什么类型的值，就调用该类型的转换函数

自动类型转换

- 自动转换为布尔值

- JavaScript 遇到预期为布尔值的地方（比如if语句的条件部分），就会将非布尔值的参数自动转换为布尔值。系统内部会自动调用Boolean函数

```
if ( !undefined
    && !null
    && !0
    && !NaN
    && !''
) {
    console.log('true');
} // true
```

自动类型转换

- 自动转换为字符串

- JavaScript 遇到预期为字符串的地方，就会将非字符串的值自动转为字符串，主要发生在字符串的加法运算时
- 具体规则是，先将复合类型的值转为原始类型的值，再将原始类型的值转为字符串

```
'5' + 1 // '51'  
'5' + true // "5true"  
'5' + false // "5false"  
'5' + {} // "5[object Object]"  
'5' + [] // "5"  
'5' + function (){} // "5function (){}"  
'5' + undefined // "5undefined"  
'5' + null // "5null"
```

自动类型转换

- 自动转换为数值

- JavaScript 遇到预期为数值的地方，会将参数值自动转换为数值，系统内部会自动调用Number函数
- 除了加法运算符 (+) 有可能把运算符转为字符串，其他运算符都会把运算符自动转成数值

```
'5' - '2' // 3
'5' * '2' // 10
true - 1 // 0
false - 1 // -1
'1' - 1 // 0
'5' * [] // 0
false / '5' // 0
'abc' - 1 // NaN
null + 1 // 1
undefined + 1 // NaN
```

```
+'abc' // NaN
-'abc' // NaN
+true // 1
-true // 0
```

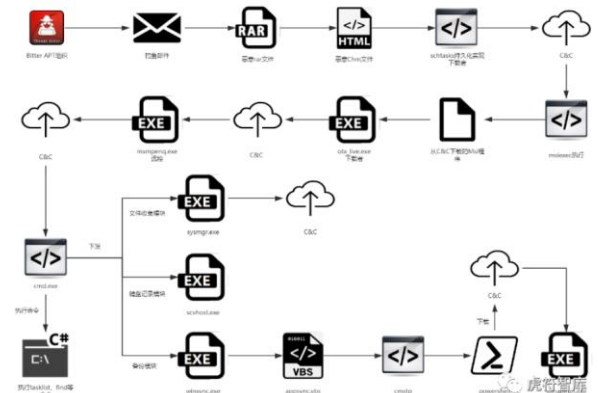
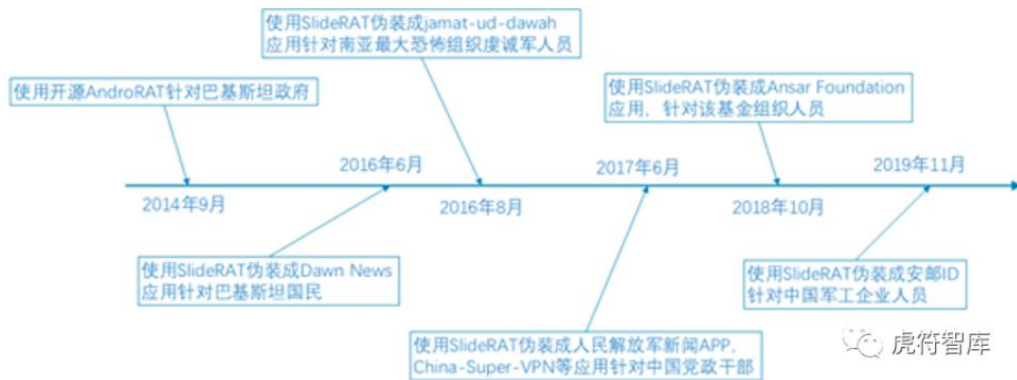
```
> null + '1'
< 'null1'
> null + 1
< 1
```

null转为数值时为0，而undefined转为数值时为NaN

印度方向APT组织介绍—蔓灵花

➤ 蔓灵花组织主要技战术特点:

- 主要攻击手法为钓鱼邮件攻击以及仿冒正常网站进行钓鱼页面攻击，同时也曾攻陷合法网站，托管恶意工具实现水坑攻击
- 具有较强社工能力，针对不同的攻击对象，定制内容完全不同的诱饵文件
- 以针对Windows系统攻击为主，偶有针对Android平台的攻击活动
- 不具备零日漏洞挖掘能力，但具备零日漏洞使用能力



印度方向APT组织介绍—蔓灵花

真实攻击案例--构造钓鱼邮件

➤ 利用新冠疫情作为话题诱饵，构造定制化的钓鱼邮件。

■ 话题内容搜集

- 钓鱼邮件话题选择了人民日报在2020年2月6日发表的一篇名为《国台办：台湾当局不应阻碍在鄂台胞返乡》的新闻稿作为诱饵文档。
- 攻击组织将此新闻内容全部复制，形成Word文档，并将其命名为“国台办.doc”



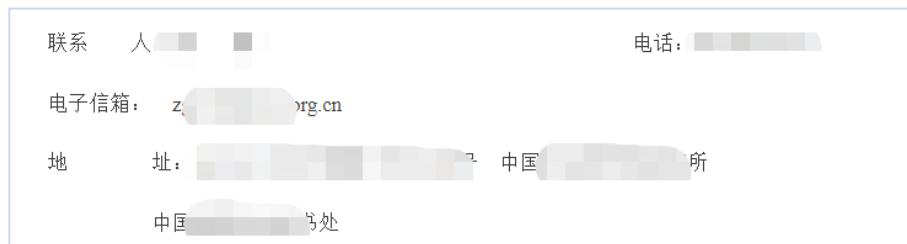
印度方向APT组织介绍—蔓灵花

真实攻击案例--构造钓鱼邮件

➤ 利用新冠疫情作为话题诱饵，构造定制化的钓鱼邮件。

■ 收、发件人信息搜集

- 根据诱饵文档有关内容，攻击者拟仿冒国台办的名义，对外发送钓鱼邮件。在国台办官网，查找到了国台办的办公地址、邮编等信息
- 根据互联网上曝光的部分密码泄露的邮箱账号信息，攻击者掌握了部分国内用的163邮箱账户名和密码，并利用 dan****@163.com、lijuan*****@163.com等邮箱，对外发送钓鱼邮件
- 收件人选择方面，攻击者选择了数个国内党政机关、科研机构作为目标单位，根据网上公开的部分联系人邮箱，共选择了近百位攻击目标个人



印度方向APT组织介绍—蔓灵花

真实攻击案例--构造钓鱼邮件

➤ 利用新冠疫情作为话题诱饵，构造定制化的钓鱼邮件。

■ 钓鱼邮件构造

- 邮件正文方面，攻击者并不了解国内人员对工作邮件的发送习惯，以惯性思维将部分英语邮件当中类似“dear”、“best regards”等问候语简单谷歌翻译成中文，阅读体验极差
- 邮件携带附件名为“国台办.zip”的压缩包，解压后得到恶意文件“国台办.exe”
- “国台办.exe”虽然为exe文件，但其图标被故意伪造成了Word文档格式，具有极强的欺骗性。如果收件人电脑选择了隐藏文件后缀名，则该文件极易被误解成一个Word文档文件。
- 恶意文件“国台办.exe”能够骗过部分杀毒软件的查杀，收件人运行时有一定概率不触发安全防护告警。至此，受害主机被攻击者完全控制。



控制

dan***@163.com

发送邮件

亲爱的同事们，

请找到附件。
最好的祝福，

中共中央台办、
国务院台办 版权所有
地址：北京市西城区广安门南街6-1号
邮编：100053

附件



国台办.zip

解压



国台办.exe

双击运行

界面显示



后台运行

